

TaskForge — Task 4: UML Diagram Portfolio

COS 214 Practical 4 · Domain: **construction site management** (Harbour Point Development)

This document accompanies the diagram files in `docs/diagrams/`. Every action, decision and object shown was taken from the final C++ source, and the runtime values in the object diagram were read off an actual run of `./taskforge`.

Figure list

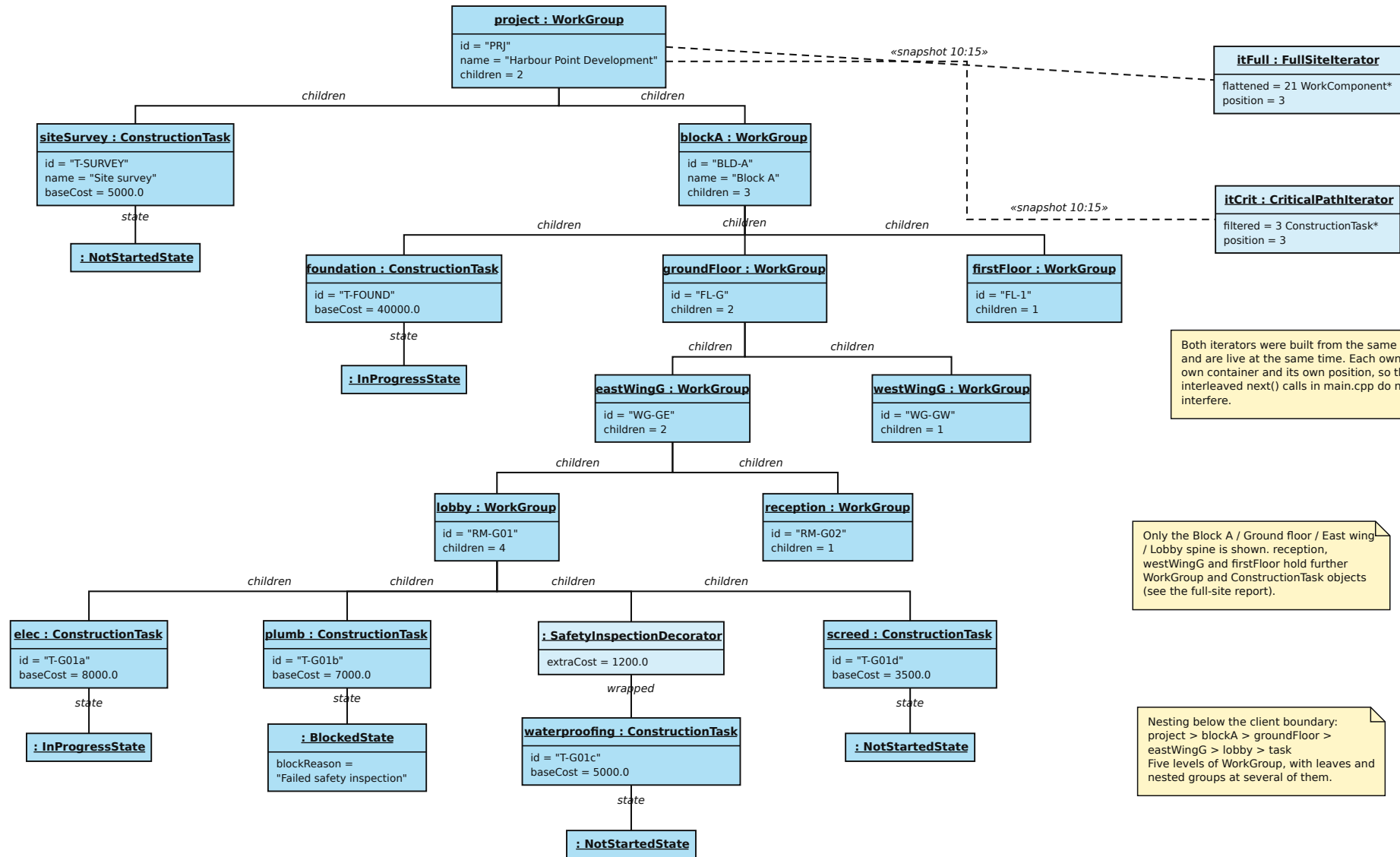
Fig.	File	What it shows
1	<code>01_object_diagram</code>	A runtime slice of the nested hierarchy at Monday 10:15, with two live iterators
2	<code>02_state_diagram</code>	Lifecycle of a <code>ConstructionTask</code>
3	<code>03_activity1_site_report</code>	Producing the site-wide status report — swimlanes , traversal made visible
3a	<code>04_activity1a_build_snapshot</code>	Expansion of the called activity <i>Build traversal snapshot</i>
4	<code>05_activity2_task_lifecycle</code>	Advancing a task at the inspection round — lifecycle-dependent branching
5	<code>06_activity3_change_and_policy</code>	Runtime change while walks are in flight — fork/join , snapshot policy

Each figure is supplied as SVG (scalable, for the PDF) and PNG at 2× (for slides or Word).

Full-size vector versions of all six figures follow immediately after this page, one per landscape sheet. They are vector art, so they stay sharp at any zoom level.

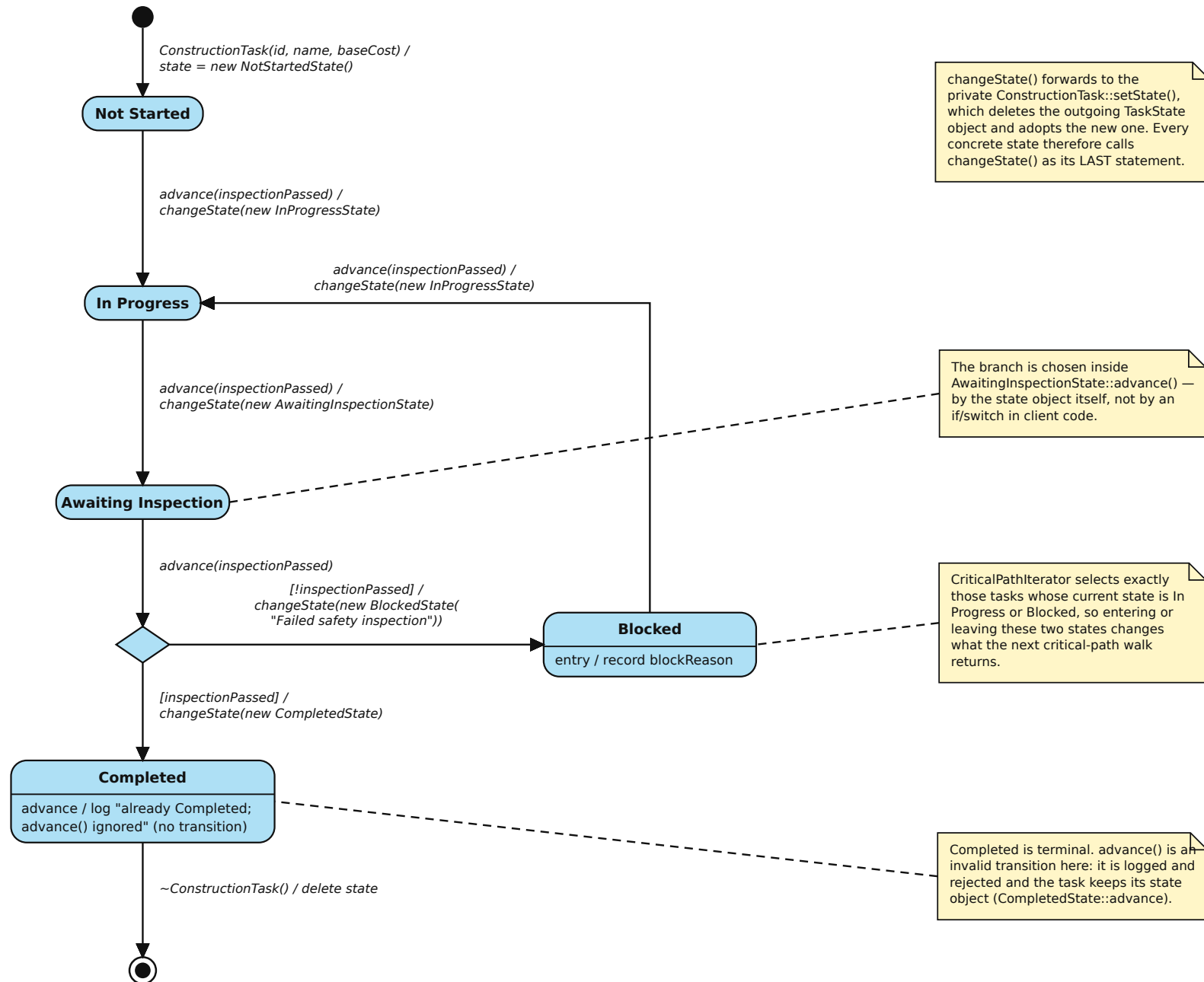
TaskForge — UML Object Diagram

Runtime snapshot of the Harbour Point site at Monday 10:15, while two independent iterators are live (main.cpp)



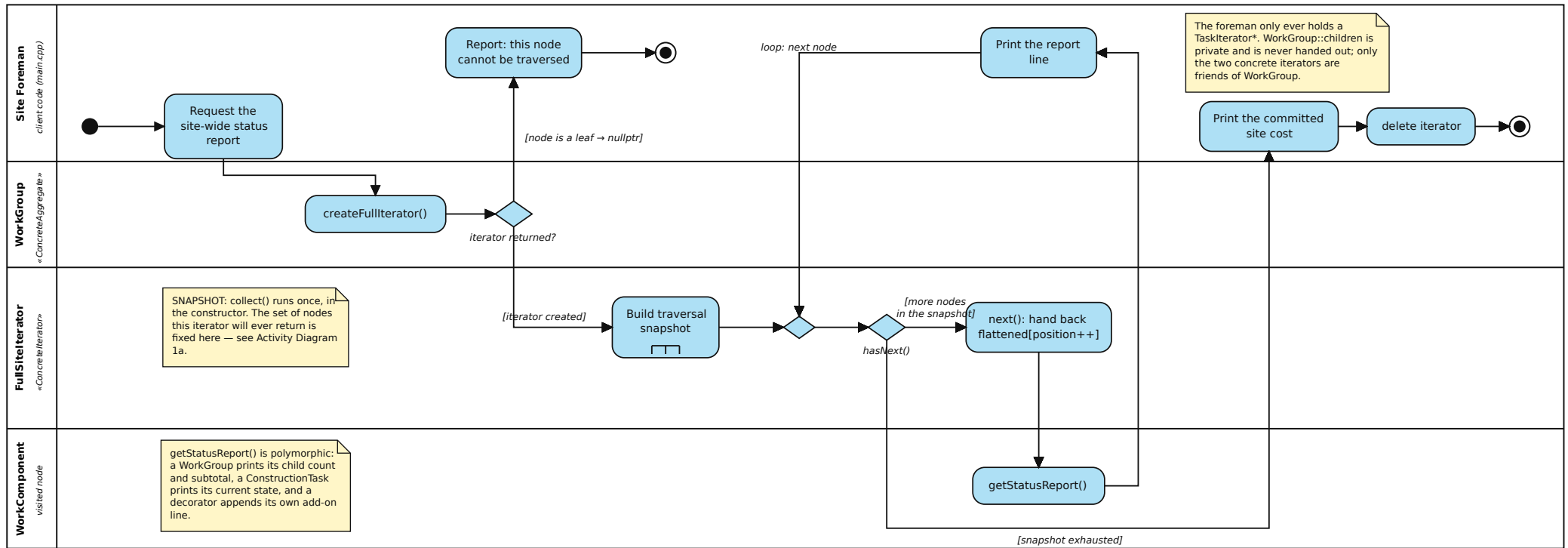
TaskForge — UML State Diagram

Lifecycle of a ConstructionTask (State pattern: Context = ConstructionTask, State = TaskState)



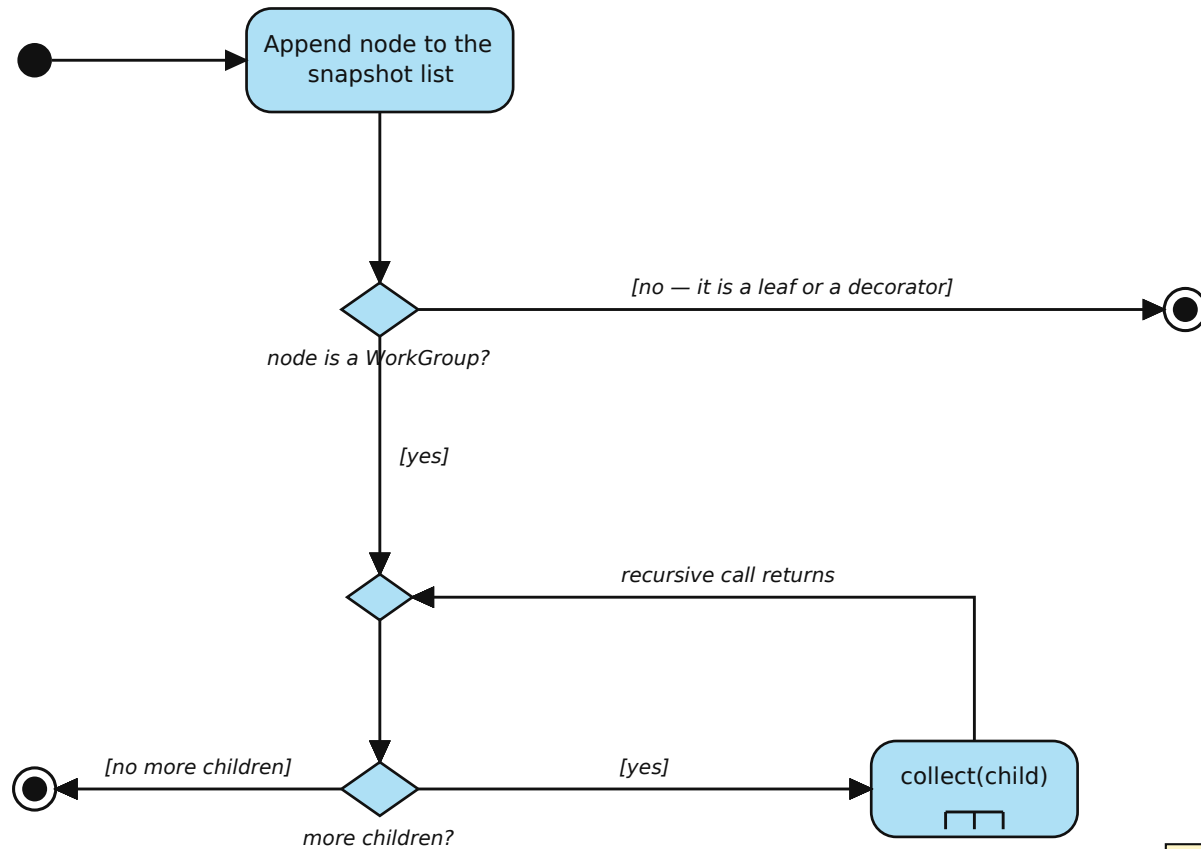
Activity Diagram 1 — Produce the site-wide status report (Monday 10:00)

Traversal is shown step by step: the loop over the iterator is part of the workflow, not hidden behind one “process all” action.



Activity Diagram 1a — called activity: “Build traversal snapshot”

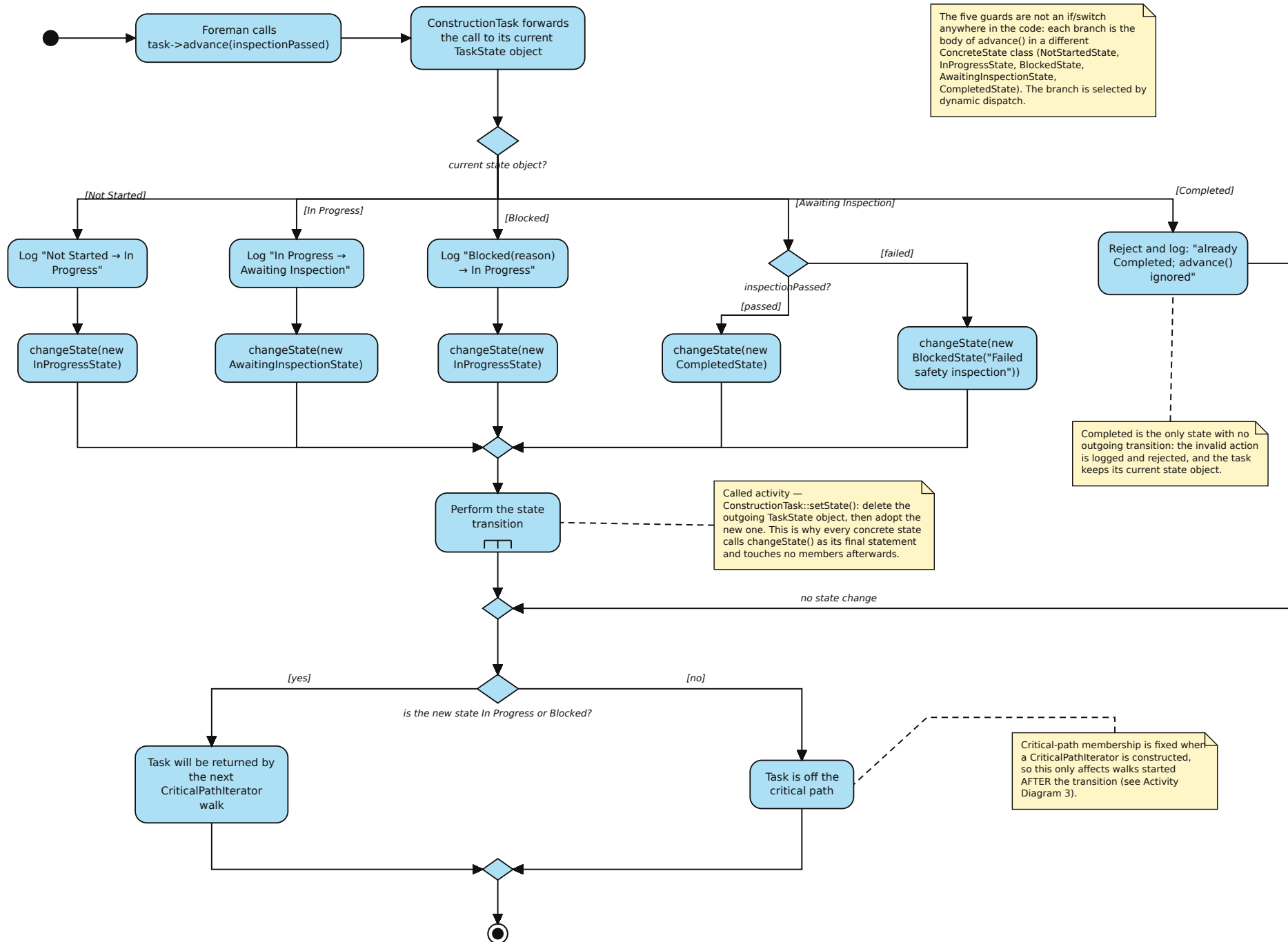
Expansion of the rake-marked action in Activity Diagram 1 (FullSiteIterator::collect)



The recursion is depth-first and pre-order, so the snapshot lists a group immediately before the sub-tree it owns. CriticalPathIterator::collect() has the same shape but appends only tasks whose state is In Progress or Blocked.

Activity Diagram 2 — Advance a task at the inspection round (Monday 14:00)

The path taken depends entirely on the task's current lifecycle state and, for one branch, on the inspection outcome.



Activity Diagram 3 — Plans change on site while walks are in flight (Monday 15:00)

The team's traversal-modification policy: a structural change and a state change happen in parallel, then the snapshot taken beforehand is replayed to prove what it does and does not see. Compresses 15:00-17:30 of main.cpp.

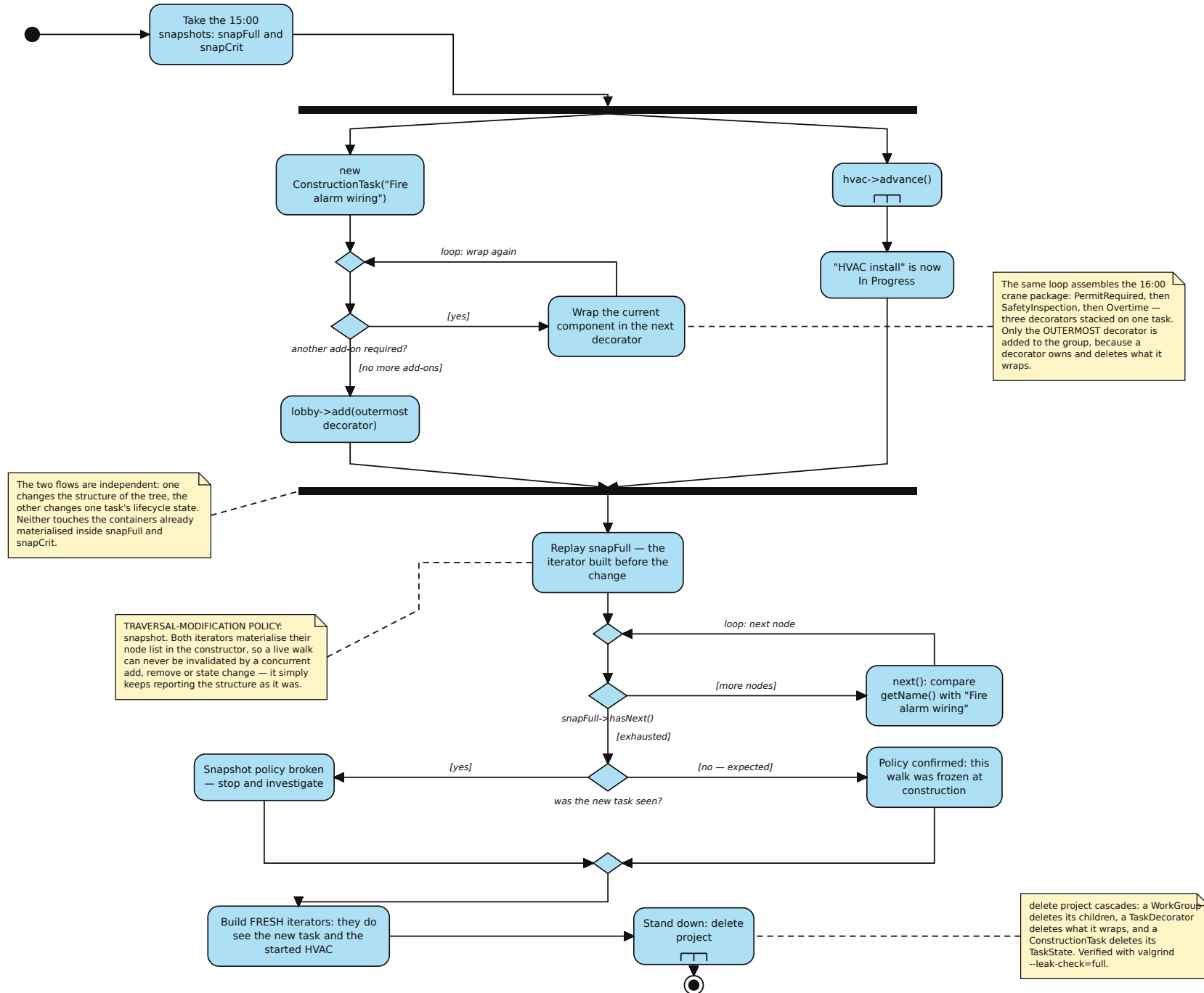


Figure 1 — Object diagram

Moment chosen: Monday 10:15, inside the "two iterators live at once" scene in `main.cpp`. This moment was picked because it is the only one in which every interesting part of the design is simultaneously observable: the recursive hierarchy, a decorated leaf, three different concrete states, and two independent iterators over the same tree.

What to read off it

- **Depth.** `project > blockA > groundFloor > eastWingG > lobby > task` — five levels of `WorkGroup` below the client boundary, comfortably past the required three. Leaf tasks appear at several levels (`siteSurvey` directly under the root, `foundation` under `blockA`, four tasks under `lobby`), so groups genuinely hold both smaller groups and individual items.
- **Decorator.** `SafetyInspectionDecorator` sits in `lobby`'s children *in place of* the task it wraps; `waterproofing` : `ConstructionTask` is reachable only through the decorator's `wrapped` link. This is the ownership rule made visible — the group holds the outermost decorator, never the raw task as well.
- **State.** Each task links to exactly one `TaskState` object. Three different concrete states are alive at this moment: `InProgressState` (`foundation`, `elec`), `BlockedState` (`plumb`, carrying `blockReason` = "Failed safety inspection"), and `NotStartedState` (the rest).
- **Iterators.** `itFull` holds 21 pointers and `itCrit` holds 3, each with its own `position` = 3. These are the real values from the program: 21 nodes in the tree at 10:15, and exactly three tasks (`foundation`, `elec`, `plumb`) that were In Progress or Blocked when `itCrit` was constructed.

The `reception`, `westWingG` and `firstFloor` sub-trees are elided to keep the figure readable; the full-site report in the program output lists them.

Figure 2 — State diagram (`ConstructionTask`)

Five states, chosen to fit construction work rather than copied from the spec's examples:

State	Class	Leaving transition
Not Started	<code>NotStartedState</code>	<code>advance()</code> → In Progress
In Progress	<code>InProgressState</code>	<code>advance()</code> → Awaiting Inspection
Awaiting Inspection	<code>AwaitingInspectionState</code>	<code>advance(true)</code> → Completed; <code>advance(false)</code> → Blocked
Blocked	<code>BlockedState</code>	<code>advance()</code> → In Progress (snag cleared)
Completed	<code>CompletedState</code>	none — terminal

Points worth defending in the demo:

- **The branch is not an if/switch.** The decision node after Awaiting Inspection is drawn as a UML decision because that is what the *behaviour* looks like, but in code it is the body of `AwaitingInspectionState::advance()`. No caller ever tests a state name.
- **The invalid transition is real, not decorative.** `advance()` on a Completed task is logged and rejected and the task keeps its state object. It is shown as an internal transition in the Completed state's compartment, and it is exercised at 14:00 in `main.cpp` ("Foreman tries to advance the finished foundation again").
- **Transition mechanics.** `changeState()` forwards to the private `ConstructionTask::setState()`, which deletes the outgoing state and adopts the new one. Because `setState()` destroys the very object executing `advance()`, every concrete state calls `changeState()` as its **last** statement and touches no members afterwards. That constraint is the single most important lifetime decision in the State implementation.
- **Link to Iterator.** In Progress and Blocked are exactly the two states `CriticalPathIterator` selects, so a transition into or out of them changes what the *next* critical-path walk returns.

Figure 3 — Activity Diagram 1: producing the site-wide status report

Why this one: it is the diagram that satisfies "at least one Activity Diagram must make traversal visible as part of a larger workflow rather than hiding the entire traversal behind a single *process all* action."

Swimlanes: Site Foreman (client code in `main.cpp`) · `WorkGroup` «ConcreteAggregate» · `FullSiteIterator` «ConcreteIterator» · `WorkComponent` (visited node). The lanes show *who is responsible* for each step, and they make the encapsulation argument for you: every action that touches the container sits in the iterator's lane, never the foreman's.

Flow. The foreman asks the root group for an iterator; the group creates one; the iterator builds its snapshot (called activity, expanded in Figure 3a). The loop is then drawn explicitly — `merge` → `hasNext()` decision → `next()` → `getStatusReport()` on the node → print the line → back to the merge. When the snapshot is exhausted the foreman prints the committed cost and deletes the iterator.

Decision with guards. `createFullIterator()` returns `nullptr` for anything that is not an aggregate (`WorkComponent::createFullIterator()`), so the diagram carries the `[node is a leaf → nullptr]` branch to an alternate ending. This is

a genuine code path, not a drawn-in condition.

Figure 3a — called activity *Build traversal snapshot*

The expansion of `FullSiteIterator::collect()`: append the node, decide whether it is a `WorkGroup`, and if so loop over its children with a recursive call. Contains a loop, two decisions with guards, a merge, and a recursive called action. The note records that `CriticalPathIterator::collect()` has the same shape but appends only tasks whose state is In Progress or Blocked.

Figure 4 — Activity Diagram 2: advancing a task at the inspection round

Why this one: it satisfies "at least one should contain meaningful conditional behaviour related to the lifecycle or runtime configuration of an object."

One decision node fans out into five mutually exclusive guarded branches — one per `ConcreteState` class — and the Awaiting Inspection branch contains a second decision on `inspectionPassed`. Four branches log and then perform a transition; the Completed branch logs a rejection and changes nothing, so it bypasses the transition entirely and rejoins at the second merge.

The tail of the diagram closes the loop back to `Iterator`: after the transition, a decision asks whether the new state is In Progress or Blocked, which determines whether the task will be picked up by the *next* `CriticalPathIterator`.

Called activity: *Perform the state transition* = `ConstructionTask::setState()`.

The note that matters. The five guards look like a switch statement and a marker will probe that. The note on the diagram states plainly that each branch is the body of `advance()` in a different class, selected by dynamic dispatch — there is no centralised `if/else` chain anywhere in `ConstructionTask` or in client code.

Figure 5 — Activity Diagram 3: plans change while walks are in flight

Why this one: it is the distinctive workflow of the domain and it documents the team's traversal-modification policy, which is the design decision the practical asks you to defend. It compresses 15:00–17:30 of `main.cpp` into one story.

Fork/join. Two snapshots are taken, then the flow forks into two independent changes:

- *Structure:* create `ConstructionTask("Fire alarm wiring")`, wrap it in decorators (loop), and `lobby->add(outermost decorator)`.
- *State:* `hvac->advance()` (a called activity — it is Figure 4), leaving "HVAC install" In Progress.

The flows are joined before the snapshot is replayed. **Be ready to defend the fork:** the two changes have no ordering constraint — one edits the tree, the other edits one task's state object, and neither touches the containers already materialised inside `snapFull` and `snapCrit`. The program happens to run them sequentially so the console output is deterministic; the fork records that nothing forces that order.

The wrapping loop. For the fire alarm it runs once (`OvertimeDecorator`). The same loop is what assembles the 16:00 crane package — `PermitRequired`, then `SafetyInspection`, then `Overtime`, three decorators stacked on one task — which is why it is drawn as a loop rather than a single action. Only the outermost decorator is added to the group, because a decorator owns and deletes what it wraps.

Replay and verdict. The 15:00 `snapFull` is replayed in a visible loop, comparing each `getName()` against "Fire alarm wiring". The expected answer is **no** — and the diagram shows the alternative branch so the check is a real test, not an assertion.

Stand-down. `delete project` is a called activity: a `WorkGroup` deletes its children, a `TaskDecorator` deletes what it wraps, and a `ConstructionTask` deletes its `TaskState` — each object exactly once, verified with `valgrind --leak-check=full`.

Traversal-modification policy (wording for the PDF)

Policy: snapshot traversal. Both `FullSiteIterator` and `CriticalPathIterator` materialise the complete list of nodes they will return inside their constructor, in `collect()`. From that moment the iterator is decoupled from the tree: adding a child, detaching a child or changing a task's state has no effect on any iterator already in existence. An in-flight walk can therefore never be invalidated, never skip a node and never dereference a dangling pointer as a result of a concurrent modification — it simply keeps reporting the structure as it was when the walk began.

Consequence, accepted deliberately. An iterator built before a change will not see that change. Client code that needs the current structure must build a fresh iterator; this is demonstrated explicitly at 15:00, where the 15:00 snapshot reports **NO** for a newly added task while a fresh iterator reports **YES**.

Why not live traversal. A live iterator holding an index into `WorkGroup::children` would be invalidated by `std::vector` reallocation on `add()` and would silently skip an element on `remove()`. Detecting that would require either modification counters on every group or handing the container out to the iterator, both of which cost more than the memory the snapshot uses. The one thing a snapshot cannot do — observe a structural change mid-walk — is not something any workflow in this system needs.

Scope note. The *set* of nodes is frozen; the *contents* are not. `next()` hands back a live `WorkComponent*`, so calling `getStatusReport()` on it reports the node's state at the time of the call, not at the time of the snapshot. This is intentional: a

report should show current states for the nodes it covers.

Notation coverage (Task 4 checklist)

Required element	Where
Actions	All three activity diagrams
Decisions and guards	Fig. 3 (<code>iterator returned?</code> , <code>hasNext()</code>); Fig. 4 (<code>current state object?</code> x5 guards, <code>inspectionPassed?</code> , critical-path test); Fig. 5 (<code>another add-on required?</code> , <code>hasNext()</code> , <code>was the new task seen?</code>)
Merges	Fig. 3 (loop merge); Fig. 4 (three merge nodes); Fig. 5 (two loop merges + outcome merge)
Loops	Fig. 3 (report loop); Fig. 3a (children loop); Fig. 5 (decorator wrapping loop, snapshot replay loop)
Fork / join	Fig. 5 (structure change state change)
Called / composite activity	Fig. 3 <i>Build traversal snapshot</i> (expanded in Fig. 3a); Fig. 4 <i>Perform the state transition</i> ; Fig. 5 <code>hvac->advance()</code> and <i>Stand down: delete project</i>
Swimlanes	Fig. 3 (four lanes)
Traversal visible in a larger workflow	Fig. 3
Conditional behaviour tied to lifecycle / runtime configuration	Fig. 4
Team's choice, distinctive workflow	Fig. 5
Object diagram of a runtime portion of the hierarchy	Fig. 1
State diagram of an important stateful object	Fig. 2

Traceability: diagram element → C++

Diagram element	Code
Fig. 1 children links	<code>WorkGroup::children</code> (private <code>std::vector<WorkComponent*></code>)
Fig. 1 wrapped link	<code>TaskDecorator::wrapped</code>
Fig. 1 state links	<code>ConstructionTask::state</code>
Fig. 1 flattened / filtered	<code>FullSiteIterator::flattened</code> , <code>CriticalPathIterator::filtered</code>
Fig. 2 all transitions	<code>NotStartedState/InProgressState/AwaitingInspectionState/BlockedState::advance()</code>
Fig. 2 internal transition on Completed	<code>CompletedState::advance()</code> — logs and returns
Fig. 2 transition action	<code>TaskState::changeState() → ConstructionTask::setState()</code>
Fig. 3 <i>Request the site-wide status report</i>	<code>printFullReport()</code> in <code>main.cpp</code>
Fig. 3 <code>createFullIterator()</code>	<code>WorkGroup::createFullIterator()</code>
Fig. 3 [node is a leaf → nullptr]	<code>WorkComponent::createFullIterator()</code> default
Fig. 3 <i>Build traversal snapshot</i>	<code>FullSiteIterator::FullSiteIterator() → collect()</code>
Fig. 3 <code>hasNext()</code> / <code>next()</code>	<code>FullSiteIterator::hasNext()</code> , <code>::next()</code>
Fig. 3 <code>getStatusReport()</code>	<code>WorkGroup</code> , <code>ConstructionTask</code> , <code>TaskDecorator</code> overrides
Fig. 3 <i>Print the committed site cost</i>	<code>WorkGroup::getCost()</code> (recursive sum)
Fig. 3a recursion	<code>FullSiteIterator::collect()</code>
Fig. 4 entry action	<code>ConstructionTask::advance(bool)</code>
Fig. 4 five guarded branches	the five <code>ConcreteState::advance()</code> bodies
Fig. 4 critical-path test	<code>CriticalPathIterator::collect()</code> state filter
Fig. 5 snapshots taken	<code>snapFull</code> / <code>snapCrit</code> construction in <code>main.cpp</code>
Fig. 5 wrapping loop	<code>OvertimeDecorator</code> on the fire alarm; <code>PermitRequired → SafetyInspection → Overtime</code> on the crane
Fig. 5 <code>lobby->add(...)</code>	<code>WorkGroup::add()</code>
Fig. 5 <code>hvac->advance()</code>	<code>ConstructionTask::advance()</code>
Fig. 5 replay loop	the <code>snapFull->hasNext()</code> loop in <code>main.cpp</code>
Fig. 5 fresh iterators	<code>fullWalkContains()</code> and <code>printCriticalPath()</code>
Fig. 5 <i>Stand down</i>	<code>~WorkGroup</code> , <code>~TaskDecorator</code> , <code>~ConstructionTask</code>

Likely demo questions, and the answers

- "Show me the Iterator participants." Iterator = `TaskIterator`; ConcreteIterators = `FullSiteIterator`, `CriticalPathIterator`; Aggregate hook = `WorkComponent::createFullIterator()` / `createCriticalPathIterator()`; ConcreteAggregate = `WorkGroup`. Client only ever holds a `TaskIterator*`.
- "How does the iterator reach the children if the vector is private?" `WorkGroup` declares the two concrete iterators as `friend`. The container is never returned, and no code in `main.cpp` can reach it.
- "You used `dynamic_cast` — isn't that a type check?" It is inside the iterator's own `collect()`, which is traversal machinery, not client-side dispatch. No client code, and no behaviour of the Composite, State or Decorator patterns, depends on it.
- "Trace this action to code." Use the traceability table above.
- "What happens to a walk in progress if I add a node?" Read the policy section above.
- "Why is the outermost decorator the thing in the group?" Because a decorator owns and deletes what it wraps; adding both the decorator and the raw task would double-free.

Regenerating or redrawing the diagrams

The SVGs are plain, hand-laid-out SVG — they open and edit in Inkscape, Illustrator, or any browser, and drop straight into LaTeX/Word at any size without pixelation. If the team would rather redraw them in Visual Paradigm so they match `class_diagram_final.png` exactly, every diagram's node list, guard text and routing is readable from the SVG and the descriptions above; the fill colour used is `#AEE0F5`, the same blue as the class diagram.